Distance Vector Routing

Distance Vector Routing (DVR) is a **dynamic routing algorithm** in which each router finds the shortest or best path to every destination using information received from its **neighboring routers .** Each router maintains a **routing table**, also called a **distance vector**, containing two important pieces of information for every destination: **(1) the best known distance/cost to reach the destination and (2) the outgoing link or next hop to use**.

The three main steps in DVR are

1. Initialization
2. Sharing
3. Updating

1. Initialization

- Each router creates its **routing table**.
- The router knows the distance to itself as **0**.
- It knows the distance to its **direct neighbors**.
- The distance to all other destinations is initially set to **∞ (infinity)**.
- The router also records the **next hop/outgoing link** for known destinations.

2. Sharing

- Each router **shares its distance vector (routing table information)** with its neighboring routers.
- The neighbors receive information about the distances that the sending router knows.
- This information is exchanged **periodically** or when there is a change in the network.

3. Updating

- After receiving a neighbor's distance vector, the router checks whether it can reach any destination through that neighbor with a **lower cost**.
- If a shorter path is found, the router **updates its distance and next hop**.
- This process continues until the routing tables become stable.

Example: Fig. 5-9. Part (a) shows a network. The first four columns of part (b) show the delay vectors received from the neighbors of router J. A claims to have a 12-msec delay to B, a 25-msec delay to C, a 40 msec delay to D, etc. Suppose that J has measured or estimated its delay to its neighbors, A, I, H, and K, as 8, 10, 12, and 6 msec, respectively.

Consider how J computes its new route to router G. It knows that it can get to A in 8 msec, and furthermore A claims to be able to get to G in 18 msec, so J knows it can count on a delay of 26 msec to G if it forwards packets bound for G to A. Similarly, it computes the delay to G via I, H, and K as 41 (31 + 10), 18 (6 + 12), and 37 (31 + 6) msec, respectively. The best of these values is 18, so it makes an entry in its routing table that the delay to G is 18 msec and that the route to use is via H. The same calculation is performed for all the other destinations, with the new routing table shown in the last column of the figure.

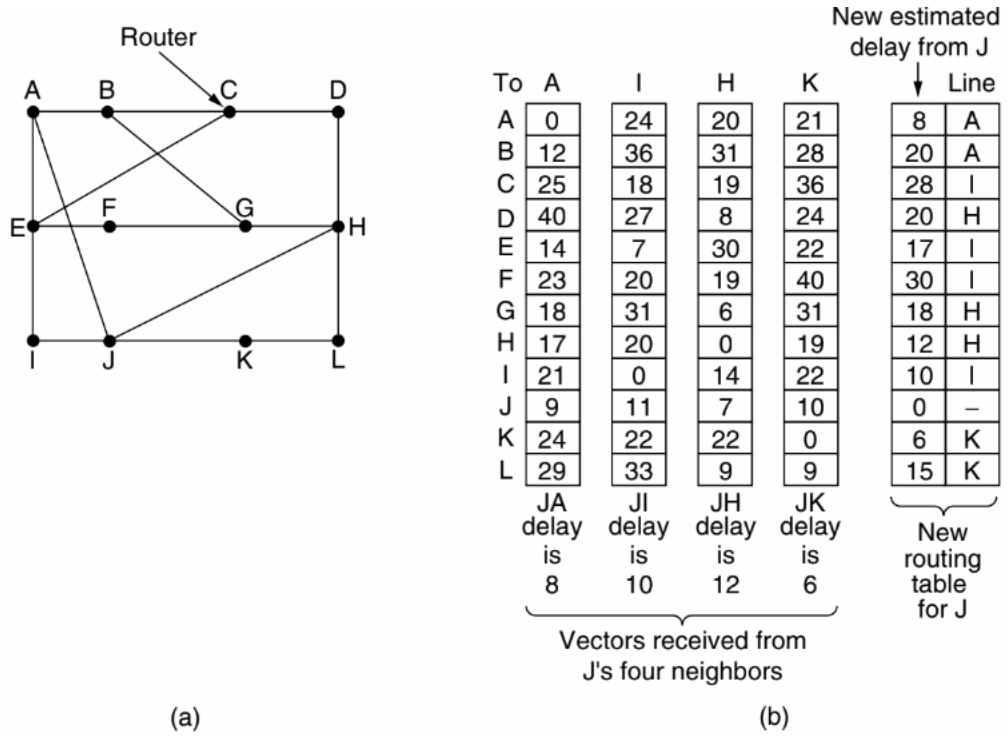


Figure 5-9. (a) A network. (b) Input from *A*, *I*, *H*, *K*, and the new routing table for *J*.

The Count-to-Infinity Problem

Convergence is the process of routers settling on the best/shortest paths across the network. Distance Vector Routing helps routers collectively calculate shortest paths. Its main drawback is slow convergence in some situations. It reacts quickly to good news (new or restored routes). It reacts slowly to bad news (failed routers or links). This slow reaction can cause problems such as the **count-to-infinity problem**.

Example:

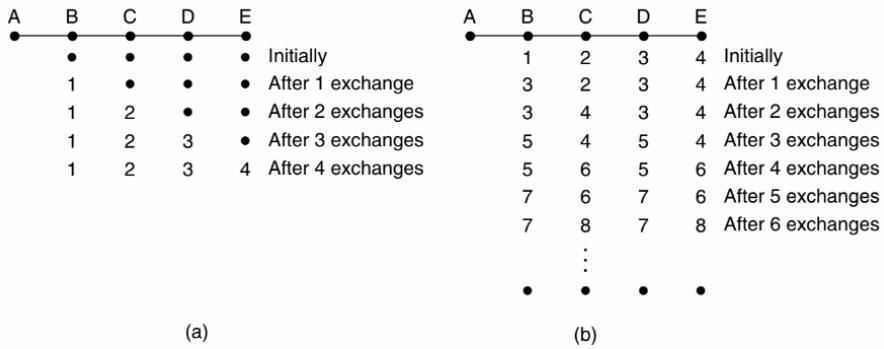


Figure 5-10. The count-to-infinity problem.

Fig 5.10(a) shows that Good news travels quickly in distance vector routing—approximately one hop per exchange.

1. Initially, **router A is down**, so all routers consider the distance to A as **infinity (∞)**.
2. When **A comes back up**, routers learn about it through periodic **vector exchanges**.
3. In the **first exchange**, B learns that A is directly connected to it → distance = **1 hop**.
4. In the **second exchange**, C learns from B that A is 1 hop away → C updates its distance to **2 hops**.
5. Then D, E, and other routers learn about A in subsequent exchanges.
6. Thus, the information about the revived router spreads **one hop per exchange**.

7. If the longest path in the network is **N hops**, then within **N exchanges**, all routers will know about the newly revived router/link.

Fig. 5-10(b) shows that **Bad news travels slowly in distance vector routing** because routers can mistakenly believe that a neighboring router still has a valid path, causing the **count-to-infinity problem**.

1. Initially, all routers are connected, with distances to **A**:
    - B = **1 hop**
    - C = **2 hops**
    - D = **3 hops**
    - E = **4 hops**
2. Suddenly, **A goes down** or the **A–B link is cut**.
3. During the first exchange, **B** does not hear from A.
4. **C tells B** that it can reach A in **2 hops**, so B assumes it can reach A through C in **3 hops**.
5. However, C's route actually goes **through B**, creating a **routing loop**.
6. In the next exchange, C sees that its neighbors claim a distance of **3 hops**, so C updates its distance to **4 hops**.
7. The distance keeps increasing with each exchange: **3 → 4 → 5 → ...**

Bad news travels slowly in distance vector routing. When a route fails, routers usually increase their distance to the destination step by step, one hop at a time, based on what their neighbors report. They keep increasing the value until they finally reach infinity.The time this takes depends on what value is chosen for infinity, so it is usually set to the longest possible path + 1. This gradual increase in distance is known as the Count-to-Infinity problem**.**

Techniques such as Split Horizon with Poisoned Reverse try to reduce the problem, but they do not Completely solve it. The main reason is that when X tells Y it has a path, Y cannot know whether Y itself is actually part of X's path, causing routing loops.